



Docker Networking, Multi-Container Validation & Data Persistence

Docker Compose + Networks + Volumes – Creative Theory Notes



1. Why Containers Couldn't Communicate

Even when multiple containers are running, they may **not be able to talk to each other** due to:

✗ Common issues:

- Containers are on **different networks**
- Containers are not configured with the same network
- Application is still using `localhost`
- Port mapping is incorrect

To enable communication, containers must share the **same Docker network** and use the **service name** to connect.



2. Creating a Custom Docker Network

Docker containers can communicate when they are on the same network.

Steps:

1. Check existing networks:

```
docker network ls
```

2. Create a new network:

```
docker network create s_network
```

The default driver is **bridge**, which works for most cases.

3. Configure containers to use the network

Both services must specify the same network in `docker-compose.yml`.



3. Rebuilding Containers to Apply Network Changes

After adding network settings:

- Stop running containers
- Rebuild using Docker Compose

This ensures:

- New network settings are applied
 - Containers can resolve each other by service name
-



4. Fixing Port Configuration

Even with correct networking, the application still failed because:

✗ Ports were misconfigured

When ports are mapped incorrectly, the host cannot access the app properly.

✓ Fix:

- Ensure both containers map ports correctly
- Use **consistent and non-conflicting host ports**

Example:

- App: 8090:8080
- Postgres: 5433:5432

After fixing ports and rebuilding, the application successfully retrieved data from the database.



5. Validating Multi-Container Setup

Successful validation shows:

- App container connects to Postgres container
- Data is retrieved correctly
- REST endpoint returns correct student data
- Containers communicate on **S_network**

This confirms the multi-container system works correctly.



6. Problem: Data Loss After Restart

The speaker tests adding new student entries.

Observed issue:

- New entries disappear after restarting containers
- Only default data remains

This demonstrates a key limitation:

! Containers are ephemeral

- Any data inside containers is lost when the container is removed
-



7. Solution: Docker Volumes for Persistence

To preserve data, Docker volumes are used.

What are volumes?

- Storage outside the container
- Stored on the host machine
- Survive container restarts and rebuilds

Why volumes matter:

- Databases require persistent storage
 - Application data should not disappear after restart
-



8. Configuring Volumes in Docker Compose

In `docker-compose.yml`:

1. Create a named volume:

volumes:

postgres_data:

2. Attach it to Postgres service:

services:

postgres:

volumes:

- postgres_data:/var/lib/postgresql/data

This maps Postgres data directory to the host volume.



9. Validating Persistence

After adding volumes:

1. Rebuild containers
2. Add new student entries
3. Restart containers
4. Check data again

Result:

- Data persists
 - Previously added entries remain
 - Container restarts no longer reset the database
-



10. Final Outcome & Learning Summary

This segment demonstrates:

✓ Docker Networking:

- Containers must share the same network
- Use service names for communication
- Port mapping must be correct

✓ Docker Volumes:

- Solve data volatility
- Provide persistent storage
- Essential for databases

✓ Real-world multi-container architecture:

- App + DB running as separate services
 - Managed using Docker Compose
 - Persistent data and network connectivity
-

If you want, I can also:

- Provide a **complete working `docker-compose.yml`**
- Show **how to connect Spring Boot to DB using service name**

- Create a **final project report / cheat sheet**
- Build a **step-by-step lab manual** for this multi-container setup